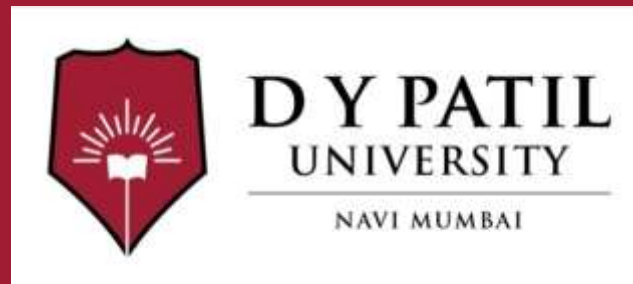


**Subject Name :Web Development with
Java**

Unit No:II

Unit Name :Java Servlet Technology

Faculty Name :Dr. Saloni Bhushan



Unit No.2

Faculty Name :

Index

Lecture No & Topic Name	Slide No

Unit No:1

Lecture No: 1



Content

Java Servlet Technology

- Introduction to Java Servlets
- J2EE architecture
- Web server and web container
- Servlet life cycle
- HTTP protocol and HTTP methods
- Types of servlets:
 - GenericServlet
 - HttpServlet
- Writing, deploying, and executing simple servlet applications
- Handling client requests and server responses (GET and POST)
- Session tracking techniques:
 - HttpSession
 - Cookies
- ServletConfig and ServletContext
- Database connectivity using Servlets



What Is a Servlet?

- A servlet is a Java class that is used to extend the capabilities of servers that host applications which can be accessed via a request-response programming model.
- Although servlets can respond to any type of request, they are commonly used to extend the applications hosted by web servers. For such applications, Java Servlet technology defines HTTP-specific servlet classes.
- The **javax.servlet** and **javax.servlet.http** packages provide interfaces and classes for writing servlets.
- All servlets must implement the **Servlet** interface, which defines life-cycle methods.
- When implementing a generic service, you can extend the **GenericServlet** class provided with the Java Servlet API.
- The **HttpServlet** class provides methods, such as `doGet` and `doPost`, for handling HTTP-specific services.

Java Servlets

- Java's answer to CGI + ASP
- A little more general than CGI/ASP, etc.
- Work with all major web servers
- Need web server servlet engine
- Need servlet development kit

What's good about them?

- Concurrency(A servlet can handle multiple request, concurrently)
- Forward request
- Portability
- Efficiency
- Safety

J2EE architecture

J2EE (now called **Java EE / Jakarta EE**) is a platform used to develop **enterprise-level, distributed, and web-based applications**.

It extends Java SE by providing:

- Web services
- Distributed computing
- Database connectivity
- Security
- Transaction management
- Component-based development

It is mainly used for:

- Banking applications
- E-commerce systems
- ERP systems
- Government portals

Multi-Tier Architecture of J2EE

J2EE follows a **Multi-Tier (N-Tier) Architecture**.

The four main tiers are:

Client Tier

Web Tier

Business Tier

Enterprise Information System (EIS) Tier

Client Tier (Presentation Client)

This is the **user interface layer**.

It can be:

- Web Browser (Chrome, Edge, etc.)
- Mobile App
- Desktop Application

Responsibilities:

- Accept user input
- Send request to server
- Display response to user

Example:

User fills a login form → Clicks submit → Request goes to Web Server

Web Tier (Presentation Logic Layer)

This tier runs on the **Web Server**.

Technologies used:

- Servlet
- JSP (Java Server Pages)
- JSF
- HTML, CSS, JavaScript

Responsibilities:

- Receive HTTPequest
- Process request
- Forward to business layer
- Generate dynamic response

Components:

- Web Container (like Apache Tomcat)

The web container:

- Manages servlets
- Handles lifecycle
- Provides security
- Manages sessions

Business Tier (Application Logic Layer)

This is the **core processing layer**.

Technologies used:

- EJB (Enterprise Java Beans)
- Spring Framework (modern applications)

Responsibilities:

- Business rules
- Validations
- Transaction management
- Decision making

Example:

- Checking login credentials
- Calculating bank interest
- Processing online payment

Runs inside:

- Application Server (GlassFish, JBoss, WebLogic)

EIS Tier (Enterprise Information System Tier)

This is the **data storage layer**.

The Data Storage Layer stores data permanently using databases.

It is responsible for saving, updating, and retrieving information such as user details, transactions, and records.

Technologies used:

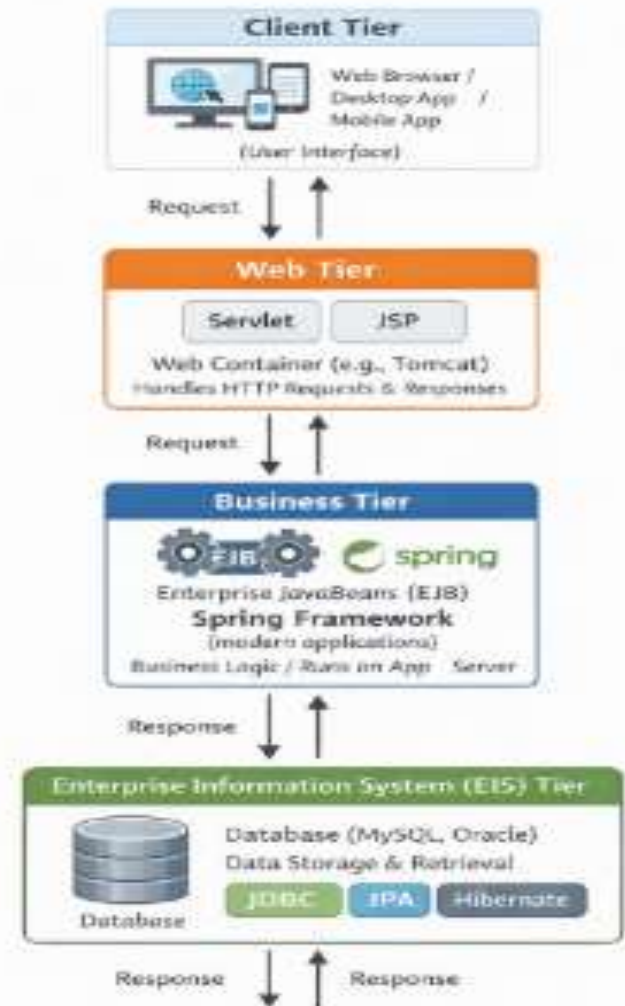
- JDBC
- JPA
- Hibernate

Complete Flow of J2EE Architecture

Let's understand the flow:

1. User sends request from Browser
2. Web Tier (Servlet/JSP) receives request
3. Web Tier calls Business Tier (EJB)
4. Business Tier accesses Database using JDBC
5. Database sends response
6. Response goes back through layers to client

J2EE Four-Tier Architecture



Web Server

A **Web Server** is software that:

- Handles **HTTP requests**
- Serves **static content**
- Communicates with the client (browser)

It works at the **presentation level**.

What Does a Web Server Do?

Receives HTTP request

Returns:

- HTML files
- CSS files
- JavaScript files
- Images
- Static content

Manages basic security

Manages connections

Examples of Web Servers

Apache HTTP Server

Microsoft IIS

Limitations

A web server **cannot execute Java code** like Servlets or JSP by itself.

It can only serve static files.

Web Container

A **Web Container** (also called Servlet Container) is a component that:

- Executes **Servlets**
- Executes **JSP**
- Manages web components

It is part of the **application server** or can run standalone (like Tomcat).

Types of Servlet

GenericServlet

- **Package:** `javax.servlet`
- **Class:** `GenericServlet`
- **Implements:** `javax.servlet.Servlet`
(indirectly implements the `Servlet` interface)
- **Type:** Protocol-independent servlet
- **Important Method:**
 - `service(ServletRequest request, ServletResponse response)`

✓ Key Points:

- Used for developing protocol-independent servlets.
- We must override the `service()` method.
- It is the parent class of `HttpServlet`.

HttpServlet

- **Package:** `javax.servlet.http`
- **Class:** `HttpServlet`
- **Extends:** `javax.servlet.GenericServlet`
- **Type:** HTTP protocol-specific servlet
- **Important Methods:**
 - `doGet(HttpServletRequest request, HttpServletResponse response)`
 - `doPost(HttpServletRequest request, HttpServletResponse response)`
 - `doPut()`
 - `doDelete()`
 - `doHead()`

✓ Key Points:

- Used for handling HTTP requests.
- We override `doGet()` or `doPost()` instead of `service()`.
- It is more commonly used in web applications.

Genericservlet example

```
import java.io.*;
import javax.servlet.*;
public class MyGenericServlet extends GenericServlet {
    @Override
    public void service(ServletRequest request, ServletResponse response)
        throws ServletException, IOException {
        // Set response content type
        response.setContentType("text/html");
        // Get writer object
        PrintWriter out = response.getWriter();
        // Write response
        out.println("<html>");
        out.println("<body>");
        out.println("<h2>Hello, this is a GenericServlet example!</h2>");
        out.println("</body>");
        out.println("</html>");
    }
}
```

Generic Servlet

1. Client sends request to:
`http://localhost:8080/YourApp/generic`
2. Web container calls the `service()` method.
3. Servlet generates HTML response.
4. `GenericServlet` is protocol-independent.
5. We must override `service()` method.
6. It is the parent class of `HttpServlet`.
7. Used when application is not strictly HTTP-based.

Basic http Servlet example

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Test extends HttpServlet{
    public void doGet(HttpServletRequest in, HttpServletResponse
out) throws ServletException, IOException {
        out.setContentType("text/html");
        PrintWriter p = out.getWriter();
        p.println("<html>");
            p.println("<body>");
                p.println("<h2>Hello, this is a HTTPServlet
example!</h2>");
                p.println("</body>");
                p.println("</html>");

    }
}
```

Servlet example

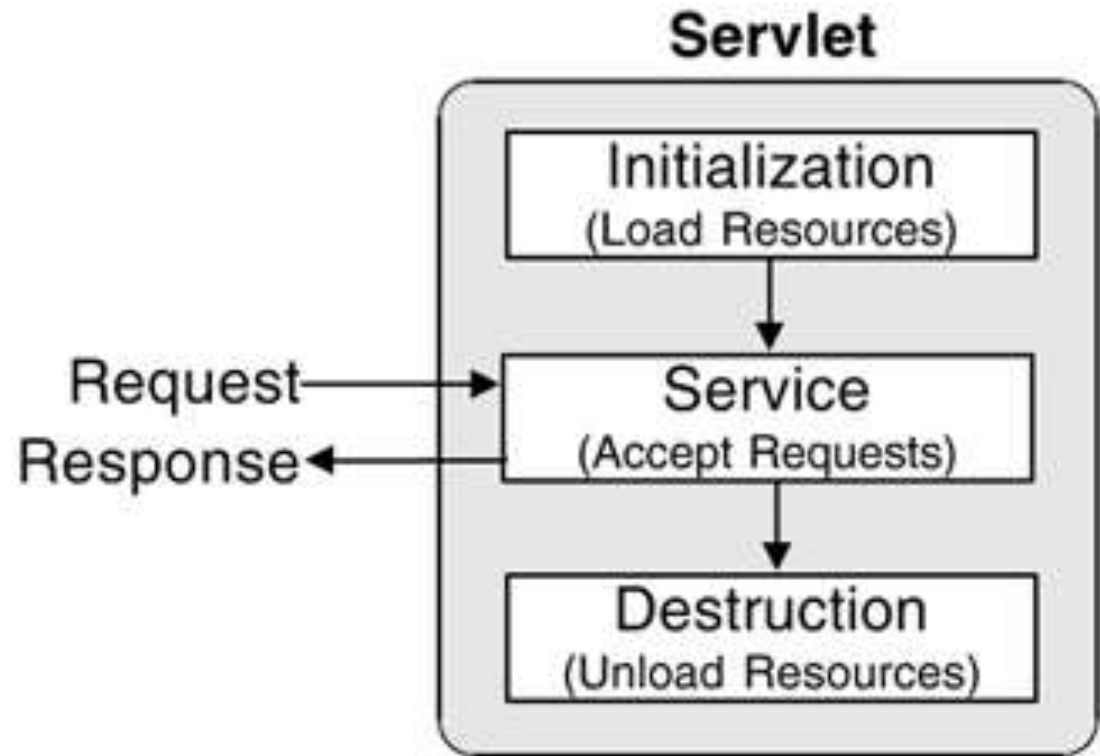
1. Client types URL in browser.
2. Browser sends GET request.
3. Web container calls `doGet()`.
4. Servlet generates HTML response.
5. Browser displays "HELLO, WORLD!".

Servlet Life Cycle

- The life cycle of a servlet is controlled by the container in which the servlet has been deployed. When a request is mapped to a servlet, the container performs the following steps.
- If an instance of the servlet does not exist, the web container Loads the servlet class.
- Creates an instance of the servlet class.
- Initializes the servlet instance by calling the **init** method.
- Invokes the **service** method, passing request and response objects.
- If the container needs to remove the servlet, it finalizes the servlet by calling the servlet's **destroy** method.

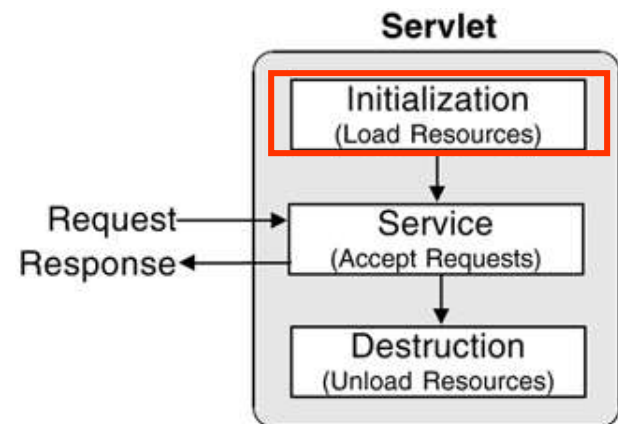
Life Cycle

- Initialize
- Service
- Destroy



Life Cycle: Initialize

- Servlet is created when servlet container receives a request from the client
public void init() throws ServletException
- `init()` method is called only once



Initializing a Servlet

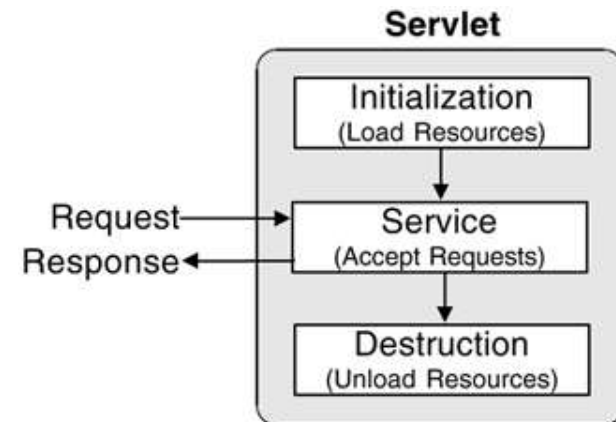
- After the web container loads and instantiates the servlet class and before it delivers requests from clients, the web container initializes the servlet.
- To customize this process to allow the servlet to read persistent configuration data, initialize resources, and perform any other one-time activities, you override the `init` method of the Servlet interface.
- A servlet that cannot complete its initialization process should throw **UnavailableException**.

Life Cycle: Service

- Any requests will be forwarded to the **service()** method

public void service(ServletRequest request, ServletResponse response) throws ServletException, IOException

- Other methods used in HttpServlet to serve client
 - doGet()
 - doPost()
 - doDelete()
 - doOptions()
 - doPut()
 - doTrace()



public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException

public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException

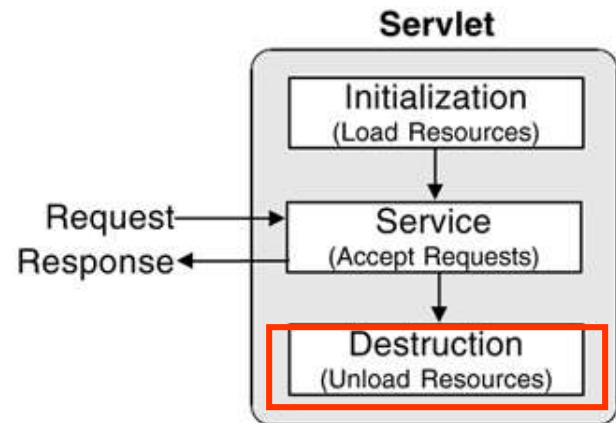
Writing Service Methods

- The service provided by a servlet is implemented in the service method of a `GenericServlet`, the `doMethod` methods (where Method can take the value Get, Delete, Options, Post, Put, Trace) of an `HttpServlet`, or any other protocol-specific methods defined by a class that implements the `Servlet` interface. the term service method will be used for any method in a servlet class that provides a service to a client.
- The general pattern for a service method is to extract information from the request, access external resources, and then populate the response based on that information.
- For HTTP servlets, the correct procedure for populating the response is to first fill in the response headers, then retrieve an output stream from the response, and finally write any body content to the output stream. Response headers must always be set before a `PrintWriter` or `ServletOutputStream` is retrieved because the HTTP protocol expects to receive all headers before body content.

Life Cycle: Destroy

- `destroy()` method is called only once
- Occurs when
 - Application is stopped
 - Servlet container shuts down
- Allows resources to be freed

public void destroy()



Servlet Life Cycle Demo

```
public class LifeCycleServlet extends HttpServlet {  
  
    public void init() {  
        System.out.println("Servlet Initialized");  
    }  
  
    public void doGet(HttpServletRequest request,  
        HttpServletResponse response)  
        throws IOException {  
        PrintWriter out=response.getWriter();  
        out.println("Service Method Called");  
    }  
  
    public void destroy() {  
        System.out.println("Servlet Destroyed");  
    }  
}
```

What is HTTP?

HTTP stands for:

HyperText Transfer Protocol

It is the protocol used for communication between:

Client (Browser) ↔ Web Server

Whenever you:

- Open a website
- Submit a form
- Click a link

HTTP is used.

Characteristics of HTTP Protocol

Application Layer protocol

Stateless protocol

Request–Response based

How HTTP Works

Step 1:

Client sends **HTTP Request**

Step 2:

Server processes request

Step 3:

Server sends **HTTP Response**

HTTP Request Structure

Request Line

Headers

Blank Line

Body (optional)

Example:

```
GET /index.html HTTP/1.1  
Host: www.example.com
```

HTTP Response Structure

Status Line

Headers

Blank Line

Body

Example:

HTTP/1.1 200 OK

Content-Type: text/html

HTTP Methods

HTTP methods define what action the client wants to perform.

GET Method-Used to retrieve data from server.

Example:Opening a webpage, Searching data

Characteristics: Data sent in URL, Visible in address bar, Limited data size, Not secure for sensitive data

Example:GET /search?name=saloni

HTTP Methods- POST Method

Used to send data to server.

Example: Login form, Registration form, Upload file.

Characteristics: Data sent in request body, Not visible in URL, No size limitation (practically large), More secure than GET

Other HTTP Methods

Method	Purpose
PUT	Replace or update an existing resource completely
DELETE	Remove a resource from the server
PATCH	Partially update an existing resource
HEAD	Retrieve only headers of a resource (no body)
OPTIONS	Retrieve the list of supported HTTP methods
TRACE	Echo the received request for debugging
CONNECT	Establish a secure tunnel to the server (used in HTTPS)

Getting Information from Requests

- A request contains data passed between a client and the servlet.
- All requests implement the **ServletRequest** interface.
- This interface defines methods for accessing all request Information.

Input to a servlet

- A **GET** request supplies parameters in the form *URL ? name=value & name=value & name=value*
 - (Illegal spaces added to make it more legible)
 - Actual spaces in the parameter values are encoded by **+** signs
 - Other special characters are encoded in hex; for example, an ampersand is represented by **%26**
- Parameter names can occur more than once, with different values
- A **POST** request supplies parameters in the same syntax, only it is in the “body” section of the request and is therefore harder for the user to see

Getting the parameters

- Input parameters are retrieved via messages to the **HttpServletRequest** object request
 - Most of the interesting methods are inherited from the superinterface **ServletRequest**
- **public Enumeration getParameterNames()**
 - Returns an **Enumeration** of the parameter names
 - If no parameters, returns an empty **Enumeration**
- **public String getParameter(String name)**
 - Returns the value of the parameter **name** as a String
 - If the parameter doesn't exist, returns **null**
 - If **name** has multiple values, only the first is returned
- **public String[] getParameterValues(name)**
 - Returns an array of values of the parameter **name**
 - If the parameter doesn't exist, returns **null**

Handling HTTP `get` Requests

- `get` request
 - Retrieve the content of a URL
- Example: **GreetServlet**
 - a servlet handles HTTP `get` requests

HTML Form

index.html

```
<html>
```

```
<body>
```

```
  <form action="GreetServlet" method="get">
```

```
    Enter Name: <input type="text"  
name="username">
```

```
    <input type="submit" value="Submit">
```

```
  </form>
```

```
</body>
```

```
</html>
```

Example of input parameters- GreetServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class GreetServlet extends HttpServlet {

    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException {
        String name = request.getParameter("username");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body>");
        out.println("<h2>Welcome " + name + "!</h2>");
        out.println("</body></html>");
    }
}
```

24.5 Handling HTTP post Requests

- HTTP post request
 - Post data from an HTML form to a server-side form handler
 - Browsers cache Web pages
- Servlet WelcomeServlet3
 - Responds to a **post** request that contains data

WelcomeServlet3 responds to a post request that contains data.

```
1  // Fig. 24.14: WelcomeServlet3.java
2  // Processing post requests containing data.
3
4  import javax.servlet.*;
5  import javax.servlet.http.*;
6  import java.io.*;
7
8  public class WelcomeServlet3 extends HttpServlet {
9
10     // process "post" request from client
11     protected void doPost( HttpServletRequest request,
12         HttpServletResponse response )
13         throws ServletException, IOException
14     {
15         String firstName = request.getParameter( "firstname" );
16
17         response.setContentType( "text/html" );
18         PrintWriter out = response.getWriter();
19
20
```

Declare a doPost method to responds to post requests.



WelcomeServlet3.java

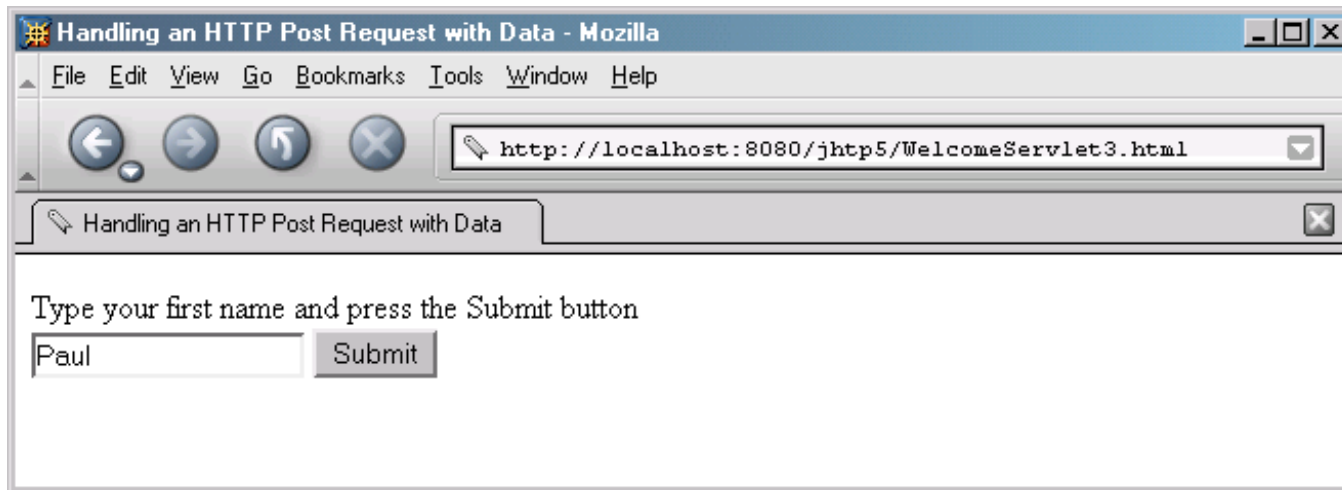
```
31     // head section of document
32     out.println( "<head>" );
33     out.println(
34         "<title>Processing post requests with data</title>" );
35     out.println( "</head>" );
36
37     // body section of document
38     out.println( "<body>" );
39     out.println( "<h1>Hello " + firstName + ",<br />" );
40     out.println( "Welcome to Servlets!</h1>" );
41     out.println( "</body>" );
42
43     // end XHTML document
44     out.println( "</html>" );
45     out.close(); // close stream to complete the page
46 }
47 }
```

HTML document in which the form's action invokes
WelcomeServlet3

```
<html>
<body>
  <form action="WelcomeServlet3"
method="post">
    Enter Name: <input type="text"
name="username">
      <input type="submit" value="Submit">
  </form>
</body>
</html>
```



Provide a form in which the user can input a name in the text input element `firstname`, then click the Submit button to invoke `WelcomeServlet3`.



Program output



Display Current Date (Server-Side Processing Demo)

```
import java.io.*;
import java.util.Date;
import javax.servlet.*;
import javax.servlet.http.*;

public class DateServlet extends HttpServlet {

    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<h2>Current Date & Time:</h2>");
        out.println(new Date());
    }
}
```

Persistent information

- A server site typically needs to maintain two kinds of **persistent** (remembered) **information**:
 - Information about the session
 - A **session** starts when the user logs in or otherwise identifies himself/herself, and continues until the user logs out or completes the transaction (for example, makes a purchase)
 - Information about the user
 - User information must generally be maintained much longer than session information (for example, remembering a purchase)
 - This information must be stored on the server, for example on a file or in a database

Summary

- A session is a continuous interaction with the user
 - HTTP is stateless, so the programmer must do something to remember session information
 - There are multiple ways to remember session information
 - The session ends when the user quits the browser (or a session may be set to time out)
- Some information must be kept longer than just within a session
 - For example, if the user orders a product, that information must be kept in a database
 - Long-term storage of information requires that the servlet have some additional privileges

Session tracking

- *HTTP is stateless*: When it gets a page request, it has *no memory* of any previous requests from the same client
 - This makes it difficult to hold a “conversation”
 - Typical example: Putting things one at a time into a shopping cart, then checking out--each page request must somehow be associated with previous requests
 - The server must be able to keep track of multiple conversations with multiple users
- *Session tracking* is keeping track of what has gone before in this particular conversation
 - Since HTTP is stateless, it does not do this for you
 - You have to do it yourself, in your servlets

Session tracking solutions

- **Java's Session Tracking API** can be used to do most of the work for you
- **Cookies** are small files that the servlet can store on the client computer, and retrieve later

The Session Tracking API

- Sessions are represented by an `HttpSession` object.
- The session tracking API is in `javax.servlet.http.HttpSession` and is built on top of cookies
- In a Servlet, an `HttpSession` object is created by calling the **`getSession()`** method on the **`HttpServletRequest`** object (named `request`). The `HttpServletRequest` object represents the client's HTTP request.
- When `request.getSession()` is called, it returns the existing session if one already exists for that client, or it creates a new session if none exists.

The Session Tracking API

- To use the session tracking API:
 - Create a session:
 - `HttpSession session = request.getSession();`
 - Returns the session associated with this request
 - If there was no associated session, one is created
 - Store information in the session and retrieve it as needed:
 - `session.setAttribute(name, value);`
 - `Object obj = session.getAttribute(name);`
- Session information is automatically maintained across requests

HttpSession Examples

- `import java.util.*;`
- `import java.io.*;`
- `import javax.servlet.*;`
- `import javax.servlet.http.*;`
- `public class DateServlet extends HttpServlet`
- `{`
- `public void`
- `doGet(HttpServletRequest request, HttpServletResponse response) throws`
- `ServletException, IOException`
- `{`
- `HttpSession`
- `hs=request.getSession(true);`
- `response.setContentType("text/html`
- `");`
- `PrintWriter`
- `pw=response.getWriter();`
- `pw.println("<HTML><HEAD><TITLE>S`
- `ession example</TITLE></HEAD>");`
- `pw.println("<BODY><b>");`
- `Date`
- `date=(Date)hs.getValue("date"`
- `);`
- `if(date!=null)`
- `{`
- `pw.println("Last access:"`
- `+date+"<br>");`
- `}`
- `date=new Date();`
- `hs.putValue("date",date);`
- `pw.println("Current date`
- `:"+date);`
- `pw.println("</BODY></HTML>`
- `");`
- `}`
- `}`

Servlet code can obtain a session by calling the getSession method thus if the following servlet is run, it will store the value of the parameter username in the session and print it out with subsequent requests to the page.

```
/** * How to use session tracking
 * with the HttpServlet interace * ***/
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class MyServlet extends HttpServlet
{
String sUserName;
protected void doGet(HttpServletRequest request, HttpServletResponse
    response) throws ServletException, IOException {
    PrintWriter out = response.getWriter();
    HttpSession session = request.getSession();
    String rUserName = request.getParameter("username");
    if(sUserName!=null){
String sUserName = (String) session.getAttribute("username");
    }else{    session.setAttribute("username",rUserName);
    } out.print("UserName="+session.getAttribute("username")) ;
    }
```

Cookies

- A **cookie** is a small bit of text sent to the client that can be read again later
 - Limitations (for the protection of the client):
 - Not more than 4KB per cookie (more than enough in general)
 - Not more than 20 cookies per site
 - Not more than 300 cookies total
- Cookies are *not* a security threat
- Cookies *can be* a privacy threat
 - Cookies can be used to customize advertisements
 - Outlook Express allows cookies to be embedded in email
 - A servlet can read your cookies

using cookies to hold states between servlet requests.

- Cookie is a file on a web user's hard disk that is used by web sites to hold information about the web users. So, the idea with cookies is to put state information into cookies and retrieve the information from the cookie in the successive requests.
- there is a specific **javax.servlet.http.Cookie** class in Servlet API.
- They can also be added and retrieved easily using methods in **HttpServletRequest** and **HttpServletResponse** objects.

Using cookies

- `import javax.servlet.http.*;`
- Constructor: `Cookie(String name, String value)`
- Assuming *request* is an `HttpServletRequest` and *response* is an `HttpServletResponse`,
 - *response*.addCookie(cookie);
 - `Cookie[ ] cookies = request.getCookies();`
 - `String name = cookies[i].getName();`
 - `String value = cookies[i].getValue();`
- There are, of course, many more methods in the `HttpServletRequest`, `HttpServletResponse`, and `Cookie` classes in the `javax.servlet.http` package



Some more Cookie methods

- `public void setComment(String purpose)`
 - `public String getComment()`
- `public void setMaxAge(int expiry)`
 - `public int getMaxAge()`
 - Max age in seconds after which cookie will expire
 - If *expiry* is negative, delete when browser exits
 - If *expiry* is zero, delete cookie immediately
- `setSecure(boolean flag)`
 - `public boolean getSecure()`
 - Indicates to the browser whether the cookie should only be sent using a secure protocol, such as HTTPS or SSL

```
protected void processRequest(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
    response.setContentType("text/html;charset=UTF-8");
    PrintWriter out = response.getWriter();

    out.println("<html>");
    out.println("<head>");
    out.println("<title>Servlet SimpleCookieServlet</title>");
    out.println("</head>");
    out.println("<body>");

    // Get cookie array from HttpServletRequest
    Cookie[] cookieArray = request.getCookies();

    // Test variable to specify whether our cookie is found or not
    boolean isCookieFound = false;
    String foundDate = "";

    if (cookieArray != null) {
        for (int i = 0; i < cookieArray.length; i++) {
            Cookie c = cookieArray[i];
            if (c.getName().equals("SimpleCookieServlet")) {
                isCookieFound = true;
                foundDate = c.getValue();
            }
        }
    }
}
```

```
if (isCookieFound) {  
    out.println("Your last visit to the site was on site on "+foundDate+".");  
}  
else {  
    out.println("This is your first visit.");  
}
```

```
// Get current date+time, create the cookie and add it
```

```
Date date = new Date();
```

```
Cookie myCookie = new Cookie ( "SimpleCookieServlet" , date.toString());  
response.addCookie (myCookie);
```

```
out.println("</body>");
```

```
out.println("</html>");
```

```
out.close();
```

```
}
```

What is web.xml?

- `web.xml` is the **deployment descriptor** of a web application.
- A **Deployment Descriptor** is an XML configuration file used to describe how a web application should be deployed and configured in a servlet container.
- Location - WEB-INF/web.xml
 - a. Defines servlet configuration
 - b. Maps URL to servlet
 - c. Stores initialization parameters
 - d. Defines context parameters

It is read when the server starts.

Why Do We Need Configuration in Servlets?

- Some data is specific to one servlet
- Some data is common to the entire web application
- How do we manage both?

How can we configure individual servlets and also share common data?

Two Levels of Data in Web Applications

Level	Example
Servlet Level	Admin email for LoginServlet
Application Level	Company Name, Visitor Count

Servlet API provides two interfaces:

- ServletConfig
- ServletContext

ServletConfig and ServletContext

In a web application, sometimes we need configuration that is specific to a single servlet, and sometimes we need data that is common to the entire application. To handle this, the Servlet API provides two interfaces:

ServletConfig and **ServletContext**.

ServletConfig is used to configure a particular servlet, while ServletContext is used to share information across the entire web application.

ServletConfig

- Interface in `javax.servlet`
- Used to configure a specific servlet
- One object per servlet
- Created during servlet initialization
- **Defined Using:** `<init-param>`
-

Key Methods:

- `getInitParameter()`
- `getServletName()`
- `getServletContext()`

Demonstrating ServletConfig

A servlet that reads an initialization parameter (admin email) specific to that servlet using `ServletConfig` and displays it in the browser.

- Servlet-specific configuration
- Uses `<init-param>`
- One ServletConfig per servlet
- **Defined Using:** `<context-param>`

web.xml Configuration

<web-app>

<servlet>

<servlet-name>ConfigServlet</servlet-name>

<servlet-class>com.test.ConfigServlet</servlet-class>

<init-param>

<param-name>adminEmail</param-name>

<param-value>admin@test.com</param-value>

</init-param>

</servlet>

<servlet-mapping>

<servlet-name>ConfigServlet</servlet-name>

<url-pattern>/config</url-pattern>

</servlet-mapping>

</web-app>

Servlet Code

```
package com.test;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ConfigServlet extends HttpServlet {

    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        ServletConfig config = getServletConfig();
        String email = config.getInitParameter("adminEmail");

        out.println("<html><body>");
        out.println("<h2>ServletConfig Example</h2>");
        out.println("<p>Admin Email: " + email + "</p>");
        out.println("</body></html>");
    }
}
```

To run

<http://localhost:8080/testWeb/config>

- 1Server starts
- Servlet is initialized
- Container creates ServletConfig object
- `<init-param>` value is stored in ServletConfig
- Browser sends request
- `doGet()` is called
- `getInitParameter()` retrieves adminEmail
- Response sent to browser

Output

ServletConfig Example

Admin Email: admin@test.com

- ✓ This parameter belongs only to ConfigServlet
- ✓ Cannot be shared with other servlets

ServletContext

- Interface in `javax.servlet`
- Used to share data across entire application
- One object per application
- Created when application starts

Key Methods:

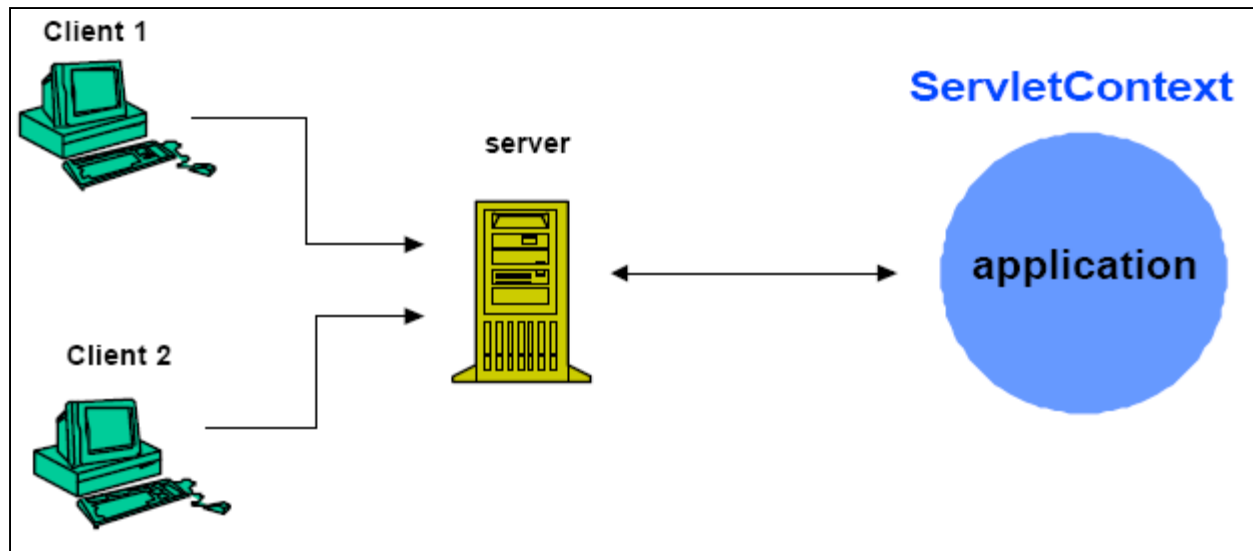
- `setAttribute()`
- `getAttribute()`
- `getInitParameter()`
- `getServerInfo()`

WebContext (**ServletContext**)

- Used by servlets to
 - Set and get context-wide (application-wide) object-valued attributes
 - Get request dispatcher
 - To forward to or include web component
 - Access Web context-wide initialization parameters set in the web.xml file
 - Access Web resources associated with the Web context
 - Log
 - Access other misc information

Scope of ServletContext

- There is **one** ServletContext object per "web application" per Java Virtual Machine



How to Access ServletContext?

- Within your servlet code, call
getServletContext()

```
public class CatalogServlet extends HttpServlet {  
    private BookDB bookDB;  
    public void init() throws ServletException {  
        // Get context-wide attribute value from  
        // ServletContext object  
        bookDB = (BookDB)getServletContext().  
                        getAttribute("bookDB");  
  
        if (bookDB == null)  
            throw new UnavailableException(  
                "Couldn't get database");  
    }  
}
```

Demonstrating ServletContext

Develop a servlet that reads a global application parameter (company name) using `ServletContext` and displays it.

- Application-wide configuration
- Uses `<context-param>`
- One ServletContext per application

web.xml Configuration

```
<web-app>
```

```
  <context-param>
```

```
    <param-name>companyName</param-name>
```

```
    <param-value>D.Y.Patil University</param-value>
```

```
  </context-param>
```

```
  <servlet>
```

```
    <servlet-name>ContextServlet</servlet-name>
```

```
    <servlet-class>com.test.ContextServlet</servlet-class>
```

```
  </servlet>
```

```
  <servlet-mapping>
```

```
    <servlet-name>ContextServlet</servlet-name>
```

```
    <url-pattern>/context</url-pattern>
```

```
  </servlet-mapping>
```

```
</web-app>
```

Servlet Code

```
package com.test;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ContextServlet extends HttpServlet {

    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        ServletContext context = getServletContext();
        String company =
        context.getInitParameter("companyName");

        out.println("<html><body>");
        out.println("<h2>ServletContext
Example</h2>");
        out.println("<p>Company Name: " + company
+ "</p>");
        out.println("</body></html>");
    }
}
```

How to Run

<http://localhost:8080/testWeb/context>

Server starts

Web application is loaded

Container creates ServletContext object

`<context-param>` is stored in ServletContext

Client sends request

`doGet()` is executed

`getInitParameter()` retrieves companyName

Response sent to browser

Output

ServletContext Example

Company Name: D.Y.Patil University

Dispatching to Another Component

```
public void doGet (HttpServletRequest request, HttpServletResponse response)
                    throws ServletException, IOException {
    HttpSession session = request.getSession(true);
    String messages = (String)session.getAttribute("messages");
    // set headers and buffer size before accessing the Writer
    response.setContentType("text/html");
    response.setBufferSize(8192);
    PrintWriter out = response.getWriter();
    // then write the response
    out.println("<html>head<title>" + messages + "</title></head></html>");
    // Get the dispatcher; it gets the banner to the user
    RequestDispatcher dispatcher =
        getServletContext().getRequestDispatcher("/banner");
    if (dispatcher != null)
        dispatcher.include(request, response);
    ...
}
```

Servlet calls other resources

- RequestDispatcher Object allows another resource to be called:

Example of calling an html file from a servlet:

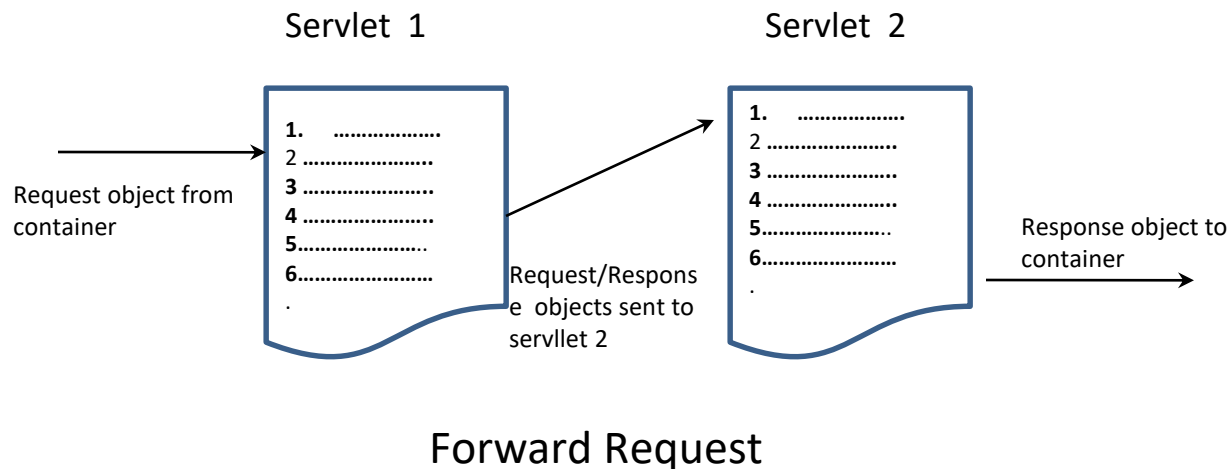
```
getServletContext().getRequestDispatcher("examples/applications/bookstore/bookstore.html");
```

Returns the html page .

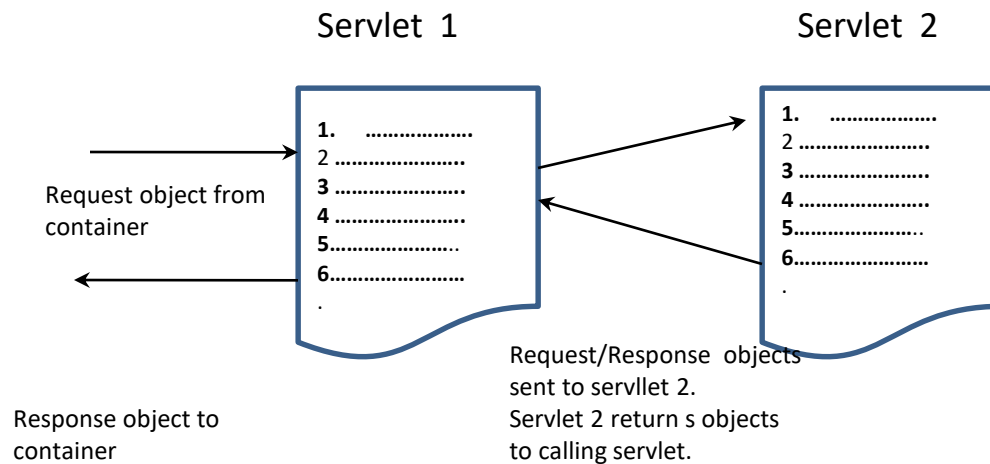
- The resources called can be
 - Another servlet
 - A Java Server Page
 - Html
 - Other

Servlet calls other resources

- Two types of requests for resources:
 - Forward request
 - Include request.



Servlet calls other resources



Include Request

Servlet calls other resources

- Forwarding using Dispatcher object:

Servlet1:

1. `HttpSession session=request.getSession(true);`
.....do something with request and response.....
2. `RequestDispatcher dispatcher=`
`getServletContext().getRequestDispatcher("/AnotherServlet");`
3. `dispatcher.forward(request, response);`

Servlet 2:

After execution of line 3 servler AnotherServlet starts execution of its doGet or doPost method. It receives the request and response obejcts with whatever information has been encapsulated, does its tasks and proceeds to write into the proper streams using the response object.

Servlet calls other resources

- Including using the Dispatcher object:

Servlet1:

1. HttpSession session=request.getSession(true);
.....do something with request and response.....
2. Dispatcher dispatcher=
 getServletContext().getRequestDispatcher("/AnotherServlet");
3. dispatcher.include(request, response);
4. Other code

Servlet 2:

Servlet AnotherServlet receives the request and response objects after execution of line 3. Adds information to the response object and finishes its tasks. Servlet 1 captures the request and response objects returned from servlet 2 at the end of line 3 execution.

It proceeds with line 4.

Servlet calls other resources

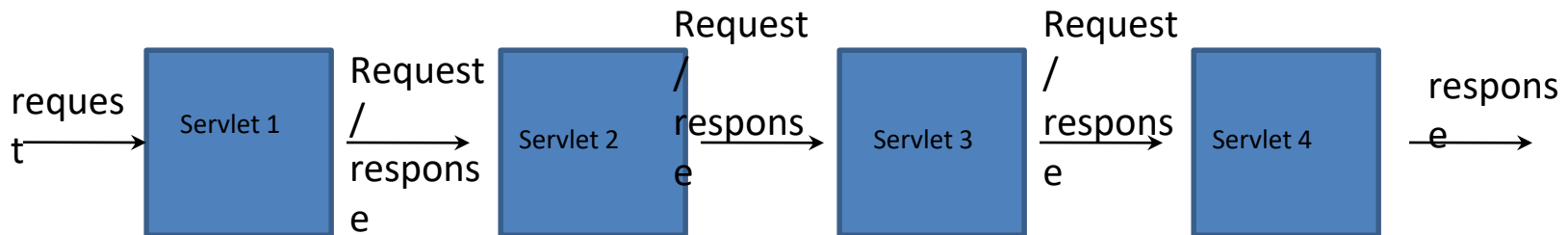
- Another approach besides the usage of the Dispatcher object is to use the redirect command:

`response.sendRedirect(String name_of_resource);`

- Sends a temporary redirect response to a client using the resource specified. The String passed in the argument can be a URL.
 - A slash / in the URL means that the resource is relative to the servlet folder root (WEB_INF/classes).
 - The response will be sent to the client by the redirected resource.

Servlet calls other resources

- A chain of servlets can be formed:



ServletConfig vs ServletContext

ServletConfig	ServletContext
One per servlet	One per application
Servlet-specific	Application-wide
Used for init parameters	Used for sharing data
Cannot share between servlets	Can share between servlets

Database connectivity with servlet

```
<form action="login" method="post">
```

```
    Username: <input type="text"
name="username"><br>
```

```
    Password: <input type="password"
name="password"><br>
```

```
    <input type="submit" value="Login">
</form>
```

```

package com.test;
import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.annotation.*;
import javax.servlet.http.*;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String user = request.getParameter("username");
        String pass = request.getParameter("password");
        try {
            // 1. Load Driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // 2. Establish Connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root",
                "Mysql@1234");
            // 3. Create Statement

```

```

PreparedStatement ps = con.prepareStatement(
    "SELECT * FROM student WHERE name=? ");
ps.setString(1, user);
//ps.setString(2, pass);
// 4. Execute Query
ResultSet rs = ps.executeQuery();
if (rs.next()) {
    out.println("<h2>Login Successful</h2>");
} else {
    out.println("<h2>Invalid Credentials</h2>");
}
// 5. Close Resources
rs.close();
ps.close();
con.close();
} catch (Exception e) {
    out.println(e);
}
}

```

Session Management

- Create a session object in your web component:

`HttpSession session = request.getSession(true);`

Either a new object is created or an existing one is returned to the servlet by the web container each time a request is made.

- You can store attribute_name/value pairs in the session object.
- There are pieces of information that you can extract using the session object:
 - `session.getID();` returns the session id for the client as a String.
 - `session.isNew();` returns true if the client has never seen a session.
 - `session.getCreationTime();` returns the time in milliseconds since the session was created.
 - `session.getLastAccessedTime();` returns the time in milliseconds since the last request generated a session.
 - `session.getMaxInactiveInterval();` returns the time in milliseconds that the session can go without access before it gets invalidated.
 - `session.getAttributeName(String name);` returns the object bound by the name in the session.
 - `session.setAttribute(String name, Object obj);` bounds an object obj by the String name. In the session

Session Management

- Example of session management:

```
HttpSession session= request.getSession(true);  
session.setAttribute("userName",request.getParameter("m  
yname"));
```

-----We store in the session object, as an object, the name retrieved from parameter myname (assuming that the user has entered a name captured by myname).

Session Management

-----Later on we check for the session:

```
session = request.getSession(true);  
if (session.getAttribute("userName") == null)  
{ session.setAttribute("userName", "Stranger"); }  
else  
PrintWriter out = response.getWriter();  
out.println("Welcome " + (String) session.getAttribute("userName") + "!!");
```

Session Management

- Another example:
- Suppose class ShoppingCart stores selected items in a shopping cart object.
- **HttpSession session = request.getSession(true);
ShoppingCart cart =
(ShoppingCart)session.getAttribute(session.getId()); // If the
user has no cart, create a new one
if (cart == null) { cart = new ShoppingCart();
session.setAttribute(session.getId(), cart); }**

The card object is associated with the particular session id associated with the client.

Session Management

- Because an object can be associated with a session, the **Duke's Bookstore** example keeps track of the books that a user has ordered within an object. The object is type `ShoppingCart` and each book that a user orders is stored in the shopping cart as a `ShoppingCartItem` object.
- Retrieving the shopping cart from the session in order to add new item in it:

```
HttpSession session = request.getSession(true);  
ShoppingCart cart =  
(ShoppingCart)session.getAttribute(session.getId());
```